Coding Assignment 1

13th September, 2026

General Instructions

- Write a separate C/C++ program for each problem. Use meaningful variable names and include comments explaining important parts of your implementation.
- Name your source files according to the question number, for example, `Q1.cpp`, `Q2.cpp`, or `Q1.c`. Each program must compile and run independently.
- Strictly follow the input and output formats specified for each problem. Read input from standard input and write output to standard output. Do not print input prompts, debugging messages, or additional text unless explicitly requested.
- Your programs can be evaluated using automated test cases, including boundary cases. Do not hard-code answers for the sample inputs. Passing the sample tests does not guarantee full marks.
- Implement the algorithms and data structures required by each question. Producing the correct output using a different method does not satisfy an explicit implementation requirement. Standard library utilities may be used unless restricted by the question, but must not replace the data structure or algorithm you are asked to implement.
- Include any experimental results or reports explicitly requested by a question. Clearly identify the corresponding question number.
- Place all source files, `README.txt`, and required reports in a folder named `yourRollNo`. Compress this folder as `yourRollNo.zip` and submit it to Google Classroom. Do not include executables, temporary files, or downloaded datasets unless requested.
- Before submission, extract your ZIP file into a new location and verify that every program compiles and runs using the instructions in `README.txt`.
- This is an individual assignment. Submit your own work. Copying code or reports from websites, peers, or other sources is prohibited. Using generative AI tools, including ChatGPT, to generate or modify submitted code or reports is also prohibited. Cite any permitted references or citation does not make prohibited copying acceptable.
- Submissions may be checked for plagiarism, and you may be asked to explain your code. Suspected violations will be reviewed under the institution's academic integrity policy. Confirmed violations may result in an F grade, as permitted by that policy.
- The submission (strict) deadline is 21st September, Monday, 10:00 PM.
- Each question consists of 10 marks.

Question:

1. Write a C++ program that constructs and modifies a Binary Search Tree (BST). Duplicate keys must not be inserted into the tree. Each node must use the following structure:

```
struct node {
    int data;
    struct node* left;
    struct node* right;
    struct node* parent;
};
```

The use of the `parent` pointer is optional. If it is used, it must be updated correctly after every insertion and deletion. You may assume that a pointer to the root node is available.

Construct the initial BST by inserting the supplied keys from left to right. The tree must be constructed by repeatedly calling `insert(root, key)`. After constructing the initial tree, print its inorder traversal, preorder traversal, and level-order traversal.

Implement and test the following functions:

(a) **Search**

```
struct node* search(struct node* root, int key);
```

The function returns a pointer to the node containing `key`. If the key is absent, it returns `nullptr`. For every search operation, print `Key found` if the key exists; otherwise, print `Key not found`.

(b) **Insertion**

```
struct node* insert(struct node* root, int key);
```

The function inserts `key` at the appropriate position and returns the updated root pointer. Perform all supplied insertions sequentially.

After every successful insertion, print the inorder traversal, preorder traversal, and the tree level by level. If the key is already present, print `Duplicate key` and leave the tree unchanged. Do not print traversals after a duplicate insertion.

(c) **Deletion**

```
struct node* deleteNode(struct node* root, int key);
```

The function deletes the node containing `key` and returns the updated root pointer. When deleting a node with two children, replace its value with the value of its inorder successor, which is the minimum-valued node in its right subtree.

Perform all supplied deletions sequentially, starting with the tree obtained after all insertion operations. After every successful deletion, print the inorder traversal, preorder traversal, and the tree level by level. If the key is absent, print `Key not found` and leave the tree unchanged. Do not print traversals after an unsuccessful deletion.

(d) **Level-order Traversal**

```
void printLevelOrder(struct node* root);
```

Print one tree level per line, from left to right. For example:

```
42
18 67
9 27 54 83
```

All keys fit in a signed 32-bit integer.

Input Format

The input consists of eight lines:

- The first line contains an integer N, the number of keys used to construct the initial BST.
- The second line contains N space-separated integers, in insertion order.
- The third line contains an integer S, the number of search operations.
- The fourth line contains S space-separated keys to search for.
- The fifth line contains an integer I, the number of additional insertion operations.
- The sixth line contains I space-separated keys to insert sequentially.
- The seventh line contains an integer D, the number of deletion operations.
- The eighth line contains D space-separated keys to delete sequentially.

Output Format

First, print the traversals of the initial tree in the following format:

```
INITIAL TREE
INORDER: <space-separated keys>
PREORDER: <space-separated keys>
LEVELORDER:
<level 1>
<level 2>
...
END
```

For each search operation, print one of the following lines:

```
SEARCH <key>: Key found
SEARCH <key>: Key not found
```

For each insertion operation, first print:

```
INSERT <key>
```

If the insertion succeeds, print a traversal block in the following format:

```
INORDER: <space-separated keys>
PREORDER: <space-separated keys>
LEVELORDER:
<level 1>
<level 2>
...
END
```

If the key is already present, print only:

```
Duplicate key
```

For each deletion operation, first print:

```
DELETE <key>
```

If the deletion succeeds, print the same traversal block shown above. If the key is absent, print only:

```
Key not found
```

Do not print trailing spaces at the end of traversal lines.

Sample Input

```
17
42 18 67 9 27 54 83 6 13 23 31 49 59 72 91 20 25
3
25 60 91
4
57 75 3 29
4
13 59 18 100
```

Sample Output

```
INITIAL TREE
INORDER: 6 9 13 18 20 23 25 27 31 42 49 54 59 67 72 83 91
PREORDER: 42 18 9 6 13 27 23 20 25 31 67 54 49 59 83 72 91
LEVELORDER:
42
18 67
9 27 54 83
6 13 23 31 49 59 72 91
20 25
END
SEARCH 25: Key found
SEARCH 60: Key not found
SEARCH 91: Key found
INSERT 57
INORDER: 6 9 13 18 20 23 25 27 31 42 49 54 57 59 67 72 83 91
PREORDER: 42 18 9 6 13 27 23 20 25 31 67 54 49 59 57 83 72 91
LEVELORDER:
42
18 67
9 27 54 83
6 13 23 31 49 59 72 91
20 25 57
END
INSERT 75
INORDER: 6 9 13 18 20 23 25 27 31 42 49 54 57 59 67 72 75 83 91
PREORDER: 42 18 9 6 13 27 23 20 25 31 67 54 49 59 57 83 72 75 91
LEVELORDER:
42
18 67
9 27 54 83
6 13 23 31 49 59 72 91
20 25 57 75
END
INSERT 3
INORDER: 3 6 9 13 18 20 23 25 27 31 42 49 54 57 59 67 72 75 83 91
PREORDER: 42 18 9 6 3 13 27 23 20 25 31 67 54 49 59 57 83 72 75 91
LEVELORDER:
42
18 67
9 27 54 83
6 13 23 31 49 59 72 91
3 20 25 57 75
END
INSERT 29
INORDER: 3 6 9 13 18 20 23 25 27 29 31 42 49 54 57 59 67 72 75 83 91
PREORDER: 42 18 9 6 3 13 27 23 20 25 31 29 67 54 49 59 57 83 72 75 91
LEVELORDER:
42
18 67
9 27 54 83
6 13 23 31 49 59 72 91
3 20 25 29 57 75
END
DELETE 13
INORDER: 3 6 9 18 20 23 25 27 29 31 42 49 54 57 59 67 72 75 83 91
PREORDER: 42 18 9 6 3 27 23 20 25 31 29 67 54 49 59 57 83 72 75 91
LEVELORDER:
42
18 67
9 27 54 83
6 23 31 49 59 72 91
```

```
3 20 25 29 57 75
END
DELETE 59
INORDER: 3 6 9 18 20 23 25 27 29 31 42 49 54 57 67 72 75 83 91
PREORDER: 42 18 9 6 3 27 23 20 25 31 29 67 54 49 57 83 72 75 91
LEVELORDER:
42
18 67
9 27 54 83
6 23 31 49 57 72 91
3 20 25 29 75
END
DELETE 18
INORDER: 3 6 9 20 23 25 27 29 31 42 49 54 57 67 72 75 83 91
PREORDER: 42 20 9 6 3 27 23 25 31 29 67 54 49 57 83 72 75 91
LEVELORDER:
42
20 67
9 27 54 83
6 23 31 49 57 72 91
3 25 29 75
END
DELETE 100
Key not found
```

In this test, deleting 13 removes a leaf node, deleting 59 removes a node with one child, deleting 18 removes a node with two children, and deleting 100 tests the case of an absent key.

2. Write a C++ program that constructs and modifies an AVL tree. An AVL tree is a binary search tree in which the absolute difference between the heights of the left and right subtrees of every node is at most one. Perform the required rotations after every successful insertion and deletion so that the tree remains balanced.

   Duplicate keys must not be inserted. Each node must use the following structure:

   ```
   struct node {
       int data;
       int height;
       struct node* left;
       struct node* right;
       struct node* parent;
   };
   ```

   The `height` and `parent` fields are optional. If either field is used, it must be updated correctly after every rotation, insertion, and deletion. You may assume that a pointer to the root node is available.

   Construct the initial AVL tree by inserting the supplied keys from left to right. The tree must be constructed by repeatedly calling `insert(root, key)`. After constructing the initial tree, print its inorder traversal, preorder traversal, and level-order traversal.

   Implement and test the following functions:

   (a) **Search**

   ```
   struct node* search(struct node* root, int key);
   ```

   The function returns a pointer to the node containing `key`. If the key is absent, it returns `nullptr`. For every search operation, print `Key found` if the key exists; otherwise, print `Key not found`.

   (b) **Insertion**

   ```
   struct node* insert(struct node* root, int key);
   ```

   The function inserts `key`, rebalances the AVL tree, and returns the updated root pointer. Perform all supplied insertions sequentially.

   After every successful insertion, print the inorder traversal, preorder traversal, and the tree level by level. If the key is already present, print `Duplicate key` and leave the tree unchanged. Do not print traversals after a duplicate insertion.

(c) **Deletion**

```
struct node* deleteNode(struct node* root, int key);
```

The name `deleteNode` is used because `delete` is a reserved keyword in C++. The function deletes the node containing `key`, rebalances the AVL tree, and returns the updated root pointer. When deleting a node with two children, replace its value with the value of its inorder successor, which is the minimum-valued node in its right subtree.

Perform all supplied deletions sequentially, starting with the tree obtained after all insertion operations. After every successful deletion, print the inorder traversal, preorder traversal, and the tree level by level. If the key is absent, print `Key not found` and leave the tree unchanged. Do not print traversals after an unsuccessful deletion.

(d) **Level-order Traversal**

```
void printLevelOrder(struct node* root);
```

Print one tree level per line, from left to right. For example:

```
31
8 78
5 21 45 90
```

All keys fit in a signed 32-bit integer.

Input Format

The input consists of eight lines:

- The first line contains an integer N, the number of keys used to construct the initial AVL tree.
- The second line contains N space-separated integers, in insertion order.
- The third line contains an integer S, the number of search operations.
- The fourth line contains S space-separated keys to search for.
- The fifth line contains an integer I, the number of additional insertion operations.
- The sixth line contains I space-separated keys to insert sequentially.
- The seventh line contains an integer D, the number of deletion operations.
- The eighth line contains D space-separated keys to delete sequentially.

Output Format

First, print the traversals of the initial tree in the following format:

```
INITIAL TREE
INORDER: <space-separated keys>
PREORDER: <space-separated keys>
LEVELORDER:
<level 1>
<level 2>
...
END
```

For each search operation, print one of the following lines:

```
SEARCH <key>: Key found
SEARCH <key>: Key not found
```

For each insertion operation, first print:

```
INSERT <key>
```

If the insertion succeeds, print:

```
INORDER: <space-separated keys>
PREORDER: <space-separated keys>
LEVELORDER:
<level 1>
<level 2>
...
END
```

If the key is already present, print only:

```
Duplicate key
```

For each deletion operation, first print:

```
DELETE <key>
```

If the deletion succeeds, print the same traversal block shown above. If the key is absent, print only:

```
Key not found
```

Do not print trailing spaces at the end of traversal lines.

Sample Input

```
17
9 8 5 4 99 78 31 34 89 90 21 23 45 77 88 112 32
3
32 56 90
4
132 156 11 7
4
7 9 31 332
```

Sample Output

```
INITIAL TREE
INORDER: 4 5 8 9 21 23 31 32 34 45 77 78 88 89 90 99 112
PREORDER: 31 8 5 4 21 9 23 78 45 34 32 77 90 89 88 99 112
LEVELORDER:
31
8 78
5 21 45 90
4 9 23 34 77 89 99
32 88 112
END
SEARCH 32: Key found
SEARCH 56: Key not found
SEARCH 90: Key found
INSERT 132
INORDER: 4 5 8 9 21 23 31 32 34 45 77 78 88 89 90 99 112 132
PREORDER: 31 8 5 4 21 9 23 78 45 34 32 77 90 89 88 112 99 132
LEVELORDER:
31
8 78
5 21 45 90
4 9 23 34 77 89 112
```

```
32 88 99 132
END
INSERT 156
INORDER: 4 5 8 9 21 23 31 32 34 45 77 78 88 89 90 99 112 132 156
PREORDER: 78 31 8 5 4 21 9 23 45 34 32 77 90 89 88 112 99 132 156
LEVELORDER:
78
31 90
8 45 89 112
5 21 34 77 88 99 132
4 9 23 32 156
END
INSERT 11
INORDER: 4 5 8 9 11 21 23 31 32 34 45 77 78 88 89 90 99 112 132 156
PREORDER: 78 31 8 5 4 21 9 11 23 45 34 32 77 90 89 88 112 99 132 156
LEVELORDER:
78
31 90
8 45 89 112
5 21 34 77 88 99 132
4 9 23 32 156
11
END
INSERT 7
INORDER: 4 5 7 8 9 11 21 23 31 32 34 45 77 78 88 89 90 99 112 132 156
PREORDER: 78 31 8 5 4 7 21 9 11 23 45 34 32 77 90 89 88 112 99 132 156
LEVELORDER:
78
31 90
8 45 89 112
5 21 34 77 88 99 132
4 7 9 23 32 156
11
END
DELETE 7
INORDER: 4 5 8 9 11 21 23 31 32 34 45 77 78 88 89 90 99 112 132 156
PREORDER: 78 31 8 5 4 21 9 11 23 45 34 32 77 90 89 88 112 99 132 156
LEVELORDER:
78
31 90
8 45 89 112
5 21 34 77 88 99 132
4 9 23 32 156
11
END
DELETE 9
INORDER: 4 5 8 11 21 23 31 32 34 45 77 78 88 89 90 99 112 132 156
PREORDER: 78 31 8 5 4 21 11 23 45 34 32 77 90 89 88 112 99 132 156
LEVELORDER:
78
31 90
8 45 89 112
5 21 34 77 88 99 132
4 11 23 32 156
END
DELETE 31
INORDER: 4 5 8 11 21 23 32 34 45 77 78 88 89 90 99 112 132 156
PREORDER: 78 32 8 5 4 21 11 23 45 34 77 90 89 88 112 99 132 156
LEVELORDER:
78
32 90
8 45 89 112
5 21 34 77 88 99 132
```

```
4 11 23 156
END
DELETE 332
Key not found
```

3. Write a C++ program that constructs and modifies a Red-Black tree. Construct the tree by processing all supplied keys from left to right and repeatedly calling `insert(root, key)`. Duplicate keys must not be inserted.

   Each internal node must use the following structure:

```
struct node {
    int data;
    struct node* left;
    struct node* right;
    struct node* parent;
    bool color;
    int black_depth;
};
```

   Use `true` for red and `false` for black. The `parent` and `black_depth` fields are optional. If they are used, they must be updated correctly after every insertion, deletion, and rotation. You may assume that a pointer to the root node is available.

   Use the standard CLRS Red-Black insertion and deletion algorithms, including their fix-up procedures. Treat every `NIL` leaf as black and do not print `NIL` leaves in any traversal. The root must always be black. These rules make the expected tree shape and coloring deterministic.

   After constructing the initial tree, print its inorder traversal, preorder traversal, and level-order traversal. Represent a node as `key(B)` if it is black and `key(R)` if it is red.

   Implement and test the following functions:

   (a) **Search**

```
struct node* search(struct node* root, int key);
```

   The function returns a pointer to the internal node containing `key`. If the key is absent, it returns `nullptr`. For every search operation, print `Key found` if the key exists; otherwise, print `Key not found`.

   (b) **Insertion**

```
struct node* insert(struct node* root, int key);
```

   The function inserts `key`, restores all Red-Black properties, and returns the updated root pointer. Perform all supplied insertions sequentially.

   After every successful insertion, print the inorder traversal, preorder traversal, and the tree level by level. If the key is already present, print `Duplicate key` and leave the tree unchanged. Do not print traversals after a duplicate insertion.

   (c) **Deletion**

```
struct node* deleteNode(struct node* root, int key);
```

   The name `deleteNode` is used because `delete` is a reserved keyword in C++. The function deletes the internal node containing `key`, applies the standard CLRS deletion fix-up, and returns the updated root pointer. When the node has two children, use its inorder successor as prescribed by the CLRS algorithm.

   Perform all supplied deletions sequentially, starting with the tree obtained after all insertion operations. After every successful deletion, print the inorder traversal, preorder traversal, and the tree level by level. If the key is absent, print `Key not found` and leave the tree unchanged. Do not print traversals after an unsuccessful deletion.

   (d) **Level-order Traversal**

```
void printLevelOrder(struct node* root);
```

   Print one level per line, from left to right. Include the color of every internal node and omit all `NIL` leaves. For example:

```
31(B)
8(B) 78(B)
5(B) 21(B) 45(R) 90(R)
```

All keys fit in a signed 32-bit integer.

Input Format

The input consists of eight lines:

- The first line contains an integer N, the number of keys used to construct the initial Red-Black tree.
- The second line contains N space-separated integers, in insertion order.
- The third line contains an integer S, the number of search operations.
- The fourth line contains S space-separated keys to search for.
- The fifth line contains an integer I, the number of additional insertion operations.
- The sixth line contains I space-separated keys to insert sequentially.
- The seventh line contains an integer D, the number of deletion operations.
- The eighth line contains D space-separated keys to delete sequentially.

Output Format

Print the initial tree as follows:

```
INITIAL TREE
INORDER: <space-separated key(color) entries>
PREORDER: <space-separated key(color) entries>
LEVELORDER:
<level 1>
<level 2>
...
END
```

For each search operation, print one of the following lines:

```
SEARCH <key>: Key found
SEARCH <key>: Key not found
```

For each insertion operation, first print `INSERT <key>`. If the insertion succeeds, print a traversal block in the same format as the initial tree, but without the `INITIAL TREE` line. If the key already exists, print only `Duplicate key` after the insertion heading.

For each deletion operation, first print `DELETE <key>`. If the deletion succeeds, print the same traversal block. If the key is absent, print only `Key not found` after the deletion heading.

Use `B` for black and `R` for red. Do not print trailing spaces at the end of traversal lines.

Sample Input

```
17
9 8 5 4 99 78 31 34 89 90 21 23 45 77 88 112 32
3
32 56 90
4
132 156 11 7
4
7 9 31 332
```

Sample Output

```
INITIAL TREE
INORDER: 4(R) 5(B) 8(B) 9(R) 21(B) 23(R) 31(B) 32(R) 34(B) 45(R) 77(B) 78(B) 88(R) 89(B) 90(R) 99(B) 112(R)
PREORDER: 31(B) 8(B) 5(B) 4(R) 21(B) 9(R) 23(R) 78(B) 45(R) 34(B) 32(R) 77(B) 90(R) 89(B) 88(R) 99(B) 112(R)
LEVELORDER:
31(B)
8(B) 78(B)
5(B) 21(B) 45(R) 90(R)
4(R) 9(R) 23(R) 34(B) 77(B) 89(B) 99(B)
32(R) 88(R) 112(R)
END
SEARCH 32: Key found
SEARCH 56: Key not found
SEARCH 90: Key found
INSERT 132
INORDER: 4(R) 5(B) 8(B) 9(R) 21(B) 23(R) 31(B) 32(R) 34(B) 45(R) 77(B) 78(B) 88(R) 89(B) 90(R) 99(R) 112(B) 132(R)
PREORDER: 31(B) 8(B) 5(B) 4(R) 21(B) 9(R) 23(R) 78(B) 45(R) 34(B) 32(R) 77(B) 90(R) 89(B) 88(R) 112(B) 99(R) 132(R)
LEVELORDER:
31(B)
8(B) 78(B)
5(B) 21(B) 45(R) 90(R)
4(R) 9(R) 23(R) 34(B) 77(B) 89(B) 112(B)
32(R) 88(R) 99(R) 132(R)
END
INSERT 156
INORDER: 4(R) 5(B) 8(B) 9(R) 21(B) 23(R) 31(B) 32(R) 34(B) 45(B) 77(B) 78(R) 88(R) 89(B) 90(B) 99(B) 112(R) 132(B) 156(R)
PREORDER: 31(B) 8(B) 5(B) 4(R) 21(B) 9(R) 23(R) 78(R) 45(B) 34(B) 32(R) 77(B) 90(B) 89(B) 88(R) 112(R) 99(B) 132(B) 156(R)
LEVELORDER:
31(B)
8(B) 78(R)
5(B) 21(B) 45(B) 90(B)
4(R) 9(R) 23(R) 34(B) 77(B) 89(B) 112(R)
32(R) 88(R) 99(B) 132(B)
156(R)
END
INSERT 11
INORDER: 4(R) 5(B) 8(B) 9(B) 11(R) 21(R) 23(B) 31(B) 32(R) 34(B) 45(B) 77(B) 78(R) 88(R) 89(B) 90(B) 99(B) 112(R) 132(B) 156(R)
PREORDER: 31(B) 8(B) 5(B) 4(R) 21(R) 9(B) 11(R) 23(B) 78(R) 45(B) 34(B) 32(R) 77(B) 90(B) 89(B) 88(R) 112(R) 99(B) 132(B) 156(R)
LEVELORDER:
31(B)
8(B) 78(R)
5(B) 21(R) 45(B) 90(B)
4(R) 9(B) 23(B) 34(B) 77(B) 89(B) 112(R)
11(R) 32(R) 88(R) 99(B) 132(B)
156(R)
END
INSERT 7
INORDER: 4(R) 5(B) 7(R) 8(B) 9(B) 11(R) 21(R) 23(B) 31(B) 32(R) 34(B) 45(B) 77(B) 78(R) 88(R) 89(B) 90(B) 99(B) 112(R) 132(B) 156(R)
PREORDER: 31(B) 8(B) 5(B) 4(R) 7(R) 21(R) 9(B) 11(R) 23(B) 78(R) 45(B) 34(B) 32(R) 77(B) 90(B) 89(B) 88(R) 112(R) 99(B) 132(B) 156(R)
LEVELORDER:
31(B)
8(B) 78(R)
5(B) 21(R) 45(B) 90(B)
4(R) 7(R) 9(B) 23(B) 34(B) 77(B) 89(B) 112(R)
11(R) 32(R) 88(R) 99(B) 132(B)
156(R)
END
DELETE 7
INORDER: 4(R) 5(B) 8(B) 9(B) 11(R) 21(R) 23(B) 31(B) 32(R) 34(B) 45(B) 77(B) 78(R) 88(R) 89(B) 90(B) 99(B) 112(R) 132(B) 156(R)
PREORDER: 31(B) 8(B) 5(B) 4(R) 21(R) 9(B) 11(R) 23(B) 78(R) 45(B) 34(B) 32(R) 77(B) 90(B) 89(B) 88(R) 112(R) 99(B) 132(B) 156(R)
LEVELORDER:
31(B)
8(B) 78(R)
5(B) 21(R) 45(B) 90(B)
4(R) 9(B) 23(B) 34(B) 77(B) 89(B) 112(R)
11(R) 32(R) 88(R) 99(B) 132(B)
156(R)
END
DELETE 9
INORDER: 4(R) 5(B) 8(B) 11(B) 21(R) 23(B) 31(B) 32(R) 34(B) 45(B) 77(B) 78(R) 88(R) 89(B) 90(B) 99(B) 112(R) 132(B) 156(R)
PREORDER: 31(B) 8(B) 5(B) 4(R) 21(R) 11(B) 23(B) 78(R) 45(B) 34(B) 32(R) 77(B) 90(B) 89(B) 88(R) 112(R) 99(B) 132(B) 156(R)
LEVELORDER:
31(B)
8(B) 78(R)
5(B) 21(R) 45(B) 90(B)
4(R) 11(B) 23(B) 34(B) 77(B) 89(B) 112(R)
32(R) 88(R) 99(B) 132(B)
156(R)
END
DELETE 31
INORDER: 4(R) 5(B) 8(B) 11(B) 21(R) 23(B) 32(B) 34(B) 45(B) 77(B) 78(R) 88(R) 89(B) 90(B) 99(B) 112(R) 132(B) 156(R)
PREORDER: 32(B) 8(B) 5(B) 4(R) 21(R) 11(B) 23(B) 78(R) 45(B) 34(B) 77(B) 90(B) 89(B) 88(R) 112(R) 99(B) 132(B) 156(R)
LEVELORDER:
32(B)
8(B) 78(R)
5(B) 21(R) 45(B) 90(B)
4(R) 11(B) 23(B) 34(B) 77(B) 89(B) 112(R)
88(R) 99(B) 132(B)
156(R)
END
DELETE 332
Key not found
```

4. Write a C++ program that constructs and modifies a 2–4 tree. A node may contain one, two, or three strictly increasing keys. An internal node with k keys has exactly $k+1$ children, and all leaves occur at the same depth. Duplicate keys must not be inserted.

   Use the following structure for each node:

```
struct Node {
    int keys[3];
    int keyCount;
    bool isLeaf;
    struct Node* children[4];
    struct Node* parent;
};
```

   The use of the `parent` pointer is optional. If it is used, it must be updated correctly after every split, borrow, merge, insertion, and deletion. You may assume that a pointer to the root node is available.

   Construct the initial tree by processing the supplied keys from left to right and repeatedly calling `insert(root, key)`. Use top-down insertion. Before descending into a full node containing three keys, split it and promote its middle key to its parent. If the root is full, create a new root and split the old root before continuing.

   After constructing the initial tree, print its inorder traversal and display the tree level by level.

   Implement and test the following functions:

   (a) **Search**

```
struct Node* search(struct Node* root, int key);
```

   The function examines every key in the current node before selecting the appropriate child. It returns a pointer to the node containing `key`. If the key is absent, it returns `nullptr`. For every search operation, print `Key found` or `Key not found`.

   (b) **Insertion**

```
struct Node* insert(struct Node* root, int key);
```

   The function performs top-down insertion and returns the updated root pointer. Search for the key before making structural changes. If the key already exists, print `Duplicate key` and leave the tree exactly unchanged.

   Perform all supplied insertions sequentially. After every successful insertion, print the inorder traversal, print the tree level by level, and call `validate(root)`. Print `VALID` if validation succeeds and `INVALID` otherwise. Do not print a traversal block after a duplicate insertion.

   (c) **Deletion**

```
struct Node* deleteKey(struct Node* root, int key);
```

   Search for the key before making structural changes. If the key is absent, print `Key not found` and leave the tree exactly unchanged.

   For a present key, use top-down deletion. Before descending into a child containing one key, ensure that the child receives an additional key. Use the following deterministic order:

   i. borrow from the left sibling if it contains at least two keys;
   ii. otherwise, borrow from the right sibling if it contains at least two keys;
   iii. otherwise, merge with the right sibling if one exists; or
   iv. if there is no right sibling, merge with the left sibling.

   When deleting a key from an internal node, replace it with its inorder predecessor if the left adjacent child contains at least two keys. Otherwise, use its inorder successor if the right adjacent child contains at least two keys. If both adjacent children contain one key, merge them with the separating key and continue in the merged node.

   After every successful deletion, print the inorder traversal, print the tree level by level, and print the result of `validate(root)`. If the root contains no keys after deletion, replace it with its only child. If the tree becomes empty, set `root` to `nullptr`.

   (d) **Validation**

```
bool validate(struct Node* root);
```

The function must verify all of the following properties:

- every node contains between one and three keys, except an empty tree;
- keys within each node are in strictly increasing order;
- an internal node containing k keys has exactly $k + 1$ children;
- every child contains only keys in the range determined by its parent;
- all leaf nodes occur at the same depth; and
- all parent pointers are correct, if parent pointers are used.

(e) **Level-order Traversal**

```
void printLevelOrder(struct Node* root);
```

Display each node using square brackets and separate keys in the same node using vertical bars. Print one level per line. For example:

```
[40]
[10|20] [60|80]
[5] [15] [30] [50] [70] [90]
```

All keys fit in a signed 32-bit integer.

Input Format

The input consists of eight lines:

- The first line contains N, the number of keys used to construct the initial tree.
- The second line contains N space-separated keys in insertion order.
- The third line contains S, the number of search operations.
- The fourth line contains S space-separated search keys.
- The fifth line contains I, the number of additional insertions.
- The sixth line contains I space-separated insertion keys.
- The seventh line contains D, the number of deletions.
- The eighth line contains D space-separated deletion keys.

Output Format

First print the initial tree:

```
INITIAL TREE
INORDER: <space-separated keys>
LEVELORDER:
<level 1>
<level 2>
...
END
```

For each search, print one of the following lines:

```
SEARCH <key>: Key found
SEARCH <key>: Key not found
```

For each insertion, first print `INSERT <key>`. If it succeeds, print:

```
INORDER: <space-separated keys>
LEVELORDER:
<level 1>
<level 2>
...
VALID
END
```

If the key is a duplicate, print only `Duplicate key` after the insertion heading.

For each deletion, first print `DELETE <key>`. If it succeeds, print the same traversal and validation block. If the key is absent, print only `Key not found` after the deletion heading. Do not print trailing spaces at the end of output lines.

Sample Input

```
23
48 22 71 9 31 58 84 5 14 27 39 53 64 77 92 2 7 12 18 25 29 36 43
3
29 60 84
4
16 61 88 96
5
2 7 31 71 500
```

Sample Output

```
INITIAL TREE
INORDER: 2 5 7 9 12 14 18 22 25 27 29 31 36 39 43 48 53 58 64 71 77 84 92
LEVELORDER:
[22|48]
[9] [31] [71]
[2|5|7] [12|14|18] [25|27|29] [36|39|43] [53|58|64] [77|84|92]
END
SEARCH 29: Key found
SEARCH 60: Key not found
SEARCH 84: Key found
INSERT 16
INORDER: 2 5 7 9 12 14 16 18 22 25 27 29 31 36 39 43 48 53 58 64 71 77 84 92
LEVELORDER:
[22|48]
[9|14] [31] [71]
[2|5|7] [12] [16|18] [25|27|29] [36|39|43] [53|58|64] [77|84|92]
VALID
END
INSERT 61
INORDER: 2 5 7 9 12 14 16 18 22 25 27 29 31 36 39 43 48 53 58 61 64 71 77 84 92
LEVELORDER:
[22|48]
[9|14] [31] [58|71]
[2|5|7] [12] [16|18] [25|27|29] [36|39|43] [53] [61|64] [77|84|92]
VALID
END
INSERT 88
INORDER: 2 5 7 9 12 14 16 18 22 25 27 29 31 36 39 43 48 53 58 61 64 71 77 84 88 92
LEVELORDER:
[22|48]
[9|14] [31] [58|71|84]
[2|5|7] [12] [16|18] [25|27|29] [36|39|43] [53] [61|64] [77] [88|92]
VALID
END
```

```
INSERT 96
INORDER: 2 5 7 9 12 14 16 18 22 25 27 29 31 36 39 43 48 53 58 61 64 71 77 84 88 92 96
LEVELORDER:
[22|48|71]
[9|14] [31] [58] [84]
[2|5|7] [12] [16|18] [25|27|29] [36|39|43] [53] [61|64] [77] [88|92|96]
VALID
END
DELETE 2
INORDER: 5 7 9 12 14 16 18 22 25 27 29 31 36 39 43 48 53 58 61 64 71 77 84 88 92 96
LEVELORDER:
[22|48|71]
[9|14] [31] [58] [84]
[5|7] [12] [16|18] [25|27|29] [36|39|43] [53] [61|64] [77] [88|92|96]
VALID
END
DELETE 7
INORDER: 5 9 12 14 16 18 22 25 27 29 31 36 39 43 48 53 58 61 64 71 77 84 88 92 96
LEVELORDER:
[22|48|71]
[9|14] [31] [58] [84]
[5] [12] [16|18] [25|27|29] [36|39|43] [53] [61|64] [77] [88|92|96]
VALID
END
DELETE 31
INORDER: 5 9 12 14 16 18 22 25 27 29 36 39 43 48 53 58 61 64 71 77 84 88 92 96
LEVELORDER:
[14|48|71]
[9] [22|29] [58] [84]
[5] [12] [16|18] [25|27] [36|39|43] [53] [61|64] [77] [88|92|96]
VALID
END
DELETE 71
INORDER: 5 9 12 14 16 18 22 25 27 29 36 39 43 48 53 58 61 64 77 84 88 92 96
LEVELORDER:
[14|48]
[9] [22|29] [58|64|84]
[5] [12] [16|18] [25|27] [36|39|43] [53] [61] [77] [88|92|96]
VALID
END
DELETE 500
Key not found
```

5. **Perfect Hashing**

   Use the **T10I4D100K** dataset available at the FIMI dataset repository. Treat all integers in the file as one stream, preserving their order. Consider the universe to be the set of positive integers.

   Implement the perfect hashing algorithm discussed in class and construct a hash table for the dataset. For each distinct element x, store the key–value pair (x, x).

   Let N be the number of elements in the stream and $q = \lceil 0.05N \rceil$. Form a set from the first q elements and generate another set containing q distinct random positive integers. Use the union of these two sets as the test set.

   Verify whether the hash table returns the correct membership result and value for each test key. Report the confusion matrix on the test set.

Input Format

For test cases, the generated random set is supplied as part of the input.

- The first line contains an integer N, the total number of elements in the dataset stream.
- The next N integers are the dataset elements in their original order. Repeated elements are allowed.
- The remaining $q = \lceil 0.05N \rceil$ integers form the random set. These integers must be distinct and positive.

```
N
a1 a2 ... aN
r1 r2 ... rq
```

Construct the hash table using only the dataset elements. Test each distinct key in the union once, in increasing order.

Output Format

For each test key, print one of the following:

```
SEARCH <key>: FOUND <value>
SEARCH <key>: NOT FOUND
```

After processing all test keys, print the confusion matrix:

```
CONFUSION_MATRIX
ACTUAL/PREDICTED PRESENT ABSENT
PRESENT <TP> <FN>
ABSENT <FP> <TN>
```

Rows indicate actual membership in the dataset, and columns indicate the result reported by the hash table.

Finally, print:

```
RESULT: CORRECT
```

if every membership result and returned value is correct; otherwise, print:

```
RESULT: INCORRECT
```

6. **Maximum Contribution of a Rank-Consistent Reporting Unit**

A company's offices form a binary reporting hierarchy. Each office stores an integer representing its monthly net contribution.

A reporting unit consists of an office and all its descendants. It is rank-consistent if every value in its left subtree is strictly smaller than its root value, every value in its right subtree is strictly greater, and both subtrees satisfy these conditions recursively.

Determine the maximum sum of any rank-consistent reporting unit. Return 0 if selecting no reporting unit gives the greatest contribution.

Input Format

- The first line contains an integer M, the number of tokens in the level-order representation.
- The next line contains M space-separated tokens. Each token is an integer or `null`, indicating a missing child.
- Starting with the root, assign subsequent tokens as the left and right children of each non-null node in queue order. Omitted trailing children are null.

Output Format

Print one integer: the maximum contribution of a rank-consistent reporting unit, or 0 if none has a positive contribution.

Constraints

- The number of non-null nodes is between 1 and 4×10^4.
- Each value lies in $[-4 \times 10^4, 4 \times 10^4]$.
- Use `long long` for contribution totals.

Sample Input

```
7
12 7 18 3 15 14 22
```

Sample Output

```
54
```

7. **Nodes at Distance K in an AVL Tree**

   Construct an AVL tree by inserting the given distinct keys from left to right using standard AVL insertion and rotations.

   Given a target key and a non-negative integer K, find all nodes exactly K edges away from the target node. Movement is allowed through left-child, right-child, and parent connections.

   Use the following structure:

```
struct Node {
    int key;
    int height;
    struct Node* left;
    struct Node* right;
};
```

   Sorting the keys and directly constructing a balanced tree is not permitted.

Input Format

- The first line contains an integer n.
- The second line contains n distinct keys in insertion order.
- The third line contains the target key and K.

Output Format

Print the keys of all nodes at distance K in increasing order, separated by spaces. Print `EMPTY` if the target is absent or no such node exists.

Constraints

- $1 \leq n \leq 10^5$.
- $-10^9 \leq \texttt{key}, \texttt{target} \leq 10^9$.
- $0 \leq K \leq n$.
- Each insertion must take $O(\log n)$ time.
- Finding the qualifying nodes must take $O(n)$ time, excluding sorting for output.

Sample Input

```
6
10 20 30 40 50 25
25 2
```

Sample Output

```
10 30
```

8. **Structural Comparison of AVL and Red-Black Trees**

Maintain an AVL tree and a Red-Black tree, both initially empty. Process the same sequence of operations on both trees from left to right:

- `I x`: insert x if it is absent.
- `D x`: delete x if it is present.

Use standard AVL insertion and deletion and standard CLRS Red-Black insertion and deletion. For deletion of a node with two children, use its inorder successor. In AVL deletion, use a single rotation when the heavy child's balance factor is zero.

After every operation, compare nodes at corresponding structural positions, identified by the same path of left and right edges from the root. Ignore Red-Black node colors.

An AVL node is matched if the Red-Black tree contains the same key at the corresponding position. Otherwise, it is ignored. Continue comparing descendant positions even if their parents differ.

Report `MATCH` only when both trees have identical structures and identical keys at every corresponding position. Otherwise, report `NO MATCH`. Calculate the ignored-node count separately after each operation.

Input Format

- The first line contains an integer Q.
- Each of the next Q lines contains an operation: `I x` or `D x`.

Output Format

After each operation, print:

```
MATCH <ignored_count> <AVL_root> <RB_root>
```

or:

```
NO MATCH <ignored_count> <AVL_root> <RB_root>
```

Print `EMPTY` for the root of an empty tree. Two empty trees produce `MATCH` with an ignored-node count of 0.

Constraints

- $1 \leq Q \leq 10^5$.
- $-10^9 \leq x \leq 10^9$.
- Duplicate insertions and deletions of absent keys leave both trees unchanged.

Sample Input

```
5
I 10
I 20
I 30
D 20
D 100
```

Sample Output

```
MATCH 0 10 10
MATCH 0 10 10
MATCH 0 20 20
MATCH 0 30 30
MATCH 0 30 30
```

9. **K-th Smallest Key in a Dynamic Binary Search Tree**

Maintain an initially empty Binary Search Tree (BST) containing distinct integer keys. Process a sequence of insertion, deletion, and k-th-smallest queries.

Support the following operations:

- `I x`: Insert key x. If it already exists, leave the tree unchanged.
- `D x`: Delete key x. If it is absent, leave the tree unchanged. When deleting a node with two children, use its inorder successor.
- `K k`: Print the k-th smallest key currently stored in the tree, where indexing starts at 1. If k exceeds the number of stored keys, print `INVALID`.

Use the following node structure:

```
struct Node {
    int key;
    int subtreeSize;
    struct Node* left;
    struct Node* right;
};
```

The field `subtreeSize` must store the number of nodes in the subtree rooted at the current node, including the node itself. Maintain this field after every insertion and deletion.

Each operation must take $O(h)$ time, where h is the current tree height. For a k-th-smallest query, do not generate the complete inorder traversal or copy the keys into an array. Tree balancing is not required.

Input Format

- The first line contains an integer Q, the number of operations.
- Each of the next Q lines contains one operation: `I x`, `D x`, or `K k`.

Output Format

For each `K k` operation, print the k-th smallest key on a separate line. Print `INVALID` if fewer than k keys are stored.

Do not print anything for insertion or deletion operations.

Constraints

- $1 \le Q \le 10^4$.
- $-10^9 \le x \le 10^9$.
- $1 \le k \le Q$.
- Duplicate insertions and deletions of absent keys are allowed.

Sample Input

```
15
I 50
I 30
I 70
I 20
I 40
I 60
I 80
K 3
D 50
K 4
I 30
K 6
D 100
K 7
K 1
```

Sample Output

```
40
60
80
INVALID
20
```

10. **Alternative Insertion Histories of a BST**

You are given a permutation of the integers from 1 to n. Insert its elements from left to right into an initially empty BST.

Determine how many other permutations construct exactly the same BST, with the same structure and the same key at every node. Do not count the original permutation.

Input Format

- The first line contains an integer n.
- The second line contains a permutation of $1, 2, \ldots, n$.

Output Format

Print the exact number of alternative insertion histories. Use arbitrary-precision integers if necessary.

Constraints

- $1 \leq n \leq 1000$.
- Every integer from 1 to n occurs exactly once.

Sample Input

```
5
3 4 5 1 2
```

Sample Output

```
5
```